

UNIVERSITATEA NAȚIONALĂ DE ȘTIINȚĂ ȘI TEHNOLOGIE
POLITEHNICA DIN BUCUREȘTI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL DE CALCULATOARE



PROIECT DE DIPLOMĂ

Identificarea referințelor lipsă din lucrări științifice

Adrian-George Dumitrache

Coordonator științific:

Conf. dr. ing. Costin-Gabriel Chiru

BUCUREȘTI

2025

NATIONAL UNIVERSITY OF SCIENCE AND TECHNOLOGY
POLITEHNICA BUCHAREST
FACULTY OF AUTOMATIC CONTROL AND COMPUTERS
COMPUTER SCIENCE AND ENGINEERING DEPARTMENT



DIPLOMA PROJECT

Identifying Missing References in Scientific Papers

Adrian-George Dumitrache

Thesis advisor:

Conf. dr. ing. Costin-Gabriel Chiru

BUCHAREST

2025

REZUMAT

Lucrările științifice folosesc citări pentru a oferi un context clar al literaturii relevante, pentru a își compara rezultatele cu alte lucrări, pentru a își defini poziția față de alte soluții, cât și pentru a acorda credit lucrărilor care au avut o influență directă asupra inovării lor. Totuși, natura manuală a procesului de citare poate duce la greșeli neintenționate, rezultând în citări incomplete ce subminează încrederea academică și provoacă confuzie. Detectarea acestor erori este dificilă și consumatoare de timp atât pentru autor, cât și pentru recenzori, ceea ce face automatizarea acestei sarcini extrem de utilă. Această lucrare analizează diverse metode pentru mitigarea problemei citărilor lipsă și propune un sistem complet de învățare automată pentru identificarea pasajelor ce necesită citare, împreună cu lucrările ce ar trebui citate în pasajele respective. În plus, soluția noastră este ușor de integrat în alte aplicații ce necesită verificare automată a integrității academice, datorită faptului că avem o singură intrare simplă (lucrarea țintă), fără a avea nevoie de feedback uman suplimentar.

ABSTRACT

Scientific papers use citations to offer accurate context of the relevant literature, compare their results to other research, define their stance to current solutions, as well as to credit the works that directly influenced their innovation. However, the manual nature of citing can lead to unintentional mistakes, resulting in incomplete citations that break academic trust and cause confusion. Detecting these errors is difficult and time consuming for the author and peer reviewers alike, making automation of this task valuable. This paper analyzes various methods for mitigating missing citations and proposes an end-to-end machine learning pipeline for identifying passages where a citation is required, as well as the paper(s) that should be cited in the respective passages. Additionally, our solution is easy to integrate in other applications that require automated verification of academic integrity due to its simple input (the target paper) and no additional human feedback being required.

CONTENTS

1	Introduction	3
1.1	Context	4
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Proposed Solution	5
1.5	Results	6
1.6	Paper Structure	6
2	Motivation	8
3	State of the Art	10
3.1	Citing Sentence Classification	11
3.1.1	Support Vector Machines and a Basic Corpus	11
3.1.2	Recurrent Convolutional Neural Networks with an Extended Corpus	11
3.1.3	Transformers, Paragraph-Level Context and an Improved Method for Labeling Scholarly Text	12
3.1.4	Comparison and Moving Forward	13
3.2	Citation Recommendation	13
3.2.1	Using Global Context	13
3.2.2	Using Local Context	14
3.2.3	Next Steps	14
3.3	Scientific Entity Recognition and Recommendation	14
4	Experiments	16
4.1	Large Language Models for Identifying Missing Citations	16
4.1.1	General Approach	16

4.1.2	Testing	17
4.1.3	Conclusions and Next Steps	18
4.2	Citation Recommendation Using Text Embeddings	18
4.2.1	Testing Scenario	19
4.2.2	Results	19
5	Proposed Solution	20
5.1	Citing Sentence Detection	21
5.2	Retrieval	23
5.3	Re-ranking	24
6	Implementation Details	27
6.1	Orchestration	28
6.2	Embeddings	28
6.3	Vector Store	29
6.4	LLM APIs	30
7	Evaluation	31
7.1	Citing Sentence Detection	31
7.2	Retrieval	33
7.3	Re-ranking	35
7.4	Execution Time Analysis	36
8	Conclusions	37
	Bibliography	38

1 INTRODUCTION

New research always builds upon already existing publications to formulate new ideas and conclusions. However, fully explaining each concept that was borrowed from the literature takes a lot of time and can even annoy already familiar readers; this is why citing other works is the main method to mark knowledge that is not part of the author's contribution, but important to it. Citing allows authors to credit other works without redundancy, to place their work in a specific academic context, and to compare results efficiently. They are also helpful for readers, empowering them to absorb any background knowledge that is necessary for understanding a paper as they need it.

Through acknowledging prior works, researchers can develop new ideas in a structured and easy-to-follow way, placing their findings in the ever-expanding [1] graph of citations. Analysis of this graph is helpful when researching specific topics, allowing readers to get an accurate timeline of developments, but can also be used for machine learning tasks using graph based models such as Graph Neural Networks [2] and Graph Transformers [3].

Citations have also become an important metric for academic careers, especially as publishing has almost entirely digitized with platforms like Google Scholar¹ or Web of Science², making summing up one's career through a simple "citation count" fairly easy. This count has led to a general "publish or perish" [4] attitude in academia with more and more people focusing on metrics to boost their scientific career, to the dismay of some [5] that claim this phenomenon has lead to lower quality publications, flooding the publishing space.

Unfortunately, citation is an error prone exercise. As more and more papers are being published [1], it is becoming increasingly hard to cite correctly, as well as keep up to date with new research. Lin et al. [6] have shown that a large number of well-accepted publications contain missing citations of named scientific entities, meaning they make basic mistakes such as mentioning the name of a machine learning model, a dataset, or an algorithm, but forget to cite the papers that introduced them. It is likely that this issue is even more prevalent outside of simple examples such as this.

Therefore, maintaining a high standard when it comes to citing is important, but is hardly trivial. The author in particular can often find it hard to assess their citations objectively. A common solution is the peer review process, which invites other researchers to read the paper and potentially identify missing citations manually. This can help catch a lot of mistakes, but is time consuming for the reviewers, making it a scarce resource.

¹<https://scholar.google.com/>

²<https://www.webofscience.com/wos/>

As a consequence, automation of this task is highly valuable and can improve the academic experience for both writers and peer reviewers. In this paper, we will explore various methods for detecting missing references in scientific papers, as well as present an end-to-end pipeline for missing citation detection and retrieval. Compared to other existing solutions, ours avoids training on specific papers, performing retrieval on indexed documents. This allows for easier scalability to a wider range of published works.

1.1 Context

Maintaining good citation practices is an error prone process that can be improved through computer assistance. The simple marking of sentences with "citation needed" can help writers improve the integrity of their papers by giving them direction when verifying their works, while paper recommendations can help authors find quality sources for their statements. This project aims to offer such a system, which can be later integrated into text editors, peer review platforms or academic integrity pipelines.

1.2 Problem Statement

Our problem can be defined as follows:

Given scientific text, return sentences whose contents require citations, as well as an associated list of papers that could be cited in that context.

While not the only valid perspective, this problem naturally breaks down into two parts:

1. Find sentences that don't seem to give enough context for their statements.
2. Retrieve candidate papers to cite for each sentence.

Our solution is modeled after these natural steps, allowing us to solve the problem in a more decoupled manner. Further sections will discuss our approach incrementally, as well as alternative interpretations that solve this problem as a unified task.

1.3 Objectives

Our main goals for this project are offering a scalable, reliable and fast system for missing reference identification that is easy to integrate into other projects that require academic integrity verification. In pursuing these goals we will attempt to improve on the current state-of-the-art.

Because the world of academic publishing moves incredibly fast, our solution needs to be capable of adapting to a large and growing number of papers which can be recommended. This is because designing and building a solution that is only capable of recommending a well defined set of papers, either due to training or architecture constraints, will naturally become obsolete as more papers come out.

Speed is also fairly key here, as integrating a slow system into interactive applications such as text editors can vastly degrade the original experience. Ideally, we'll achieve this without losing out on performance compared to state-of-the-art solutions, but some tradeoffs are acceptable.

1.4 Proposed Solution

Our solution is modeled after the steps proposed in section 1.2. The steps of our pipeline are as follows:

1. To identify sentences with missing references, we use a text classifier to classify each sentence of the target paper as either citing or non-citing. Only citing sentences progress to the next step of the pipeline.
2. For each citing sentence, we perform a fast retrieval of a set of $k = 50$ candidate papers from a database of paper titles and abstracts. The retrieval is based on semantic similarity between the statement coupled with a natural language description of the task at hand and the cited paper text.
3. Every list of candidate papers is re-ranked using a slower, but more effective ranking method to improve the order quality. Only the top $h = 5$ results from the re-ranked lists are kept and proposed as references that could be cited for the given statements.

At a high level, it is simply a combination between specialized classification used to identify potential queries that are then used to perform information retrieval on a knowledge base using a method that is reminiscent of retrieval-augmented generation [7] style approaches.

This approach allows our system to scale to a larger number of papers that can be offered as candidates, as expanding the knowledge base simply involves indexing more papers. This is way more flexible compared to neural network [8] or topic modeling approaches [9] which need to be re-trained if the set of possible recommendations changes.

Other citation recommendation systems [9] also require additional user input, such as manually pointing out passages with missing citations or placing placeholder citation markers. Our solution doesn't require any additional input from the user besides the target paper, making integration fairly simple.

1.5 Results

We present an end-to-end pipeline based on machine learning and information retrieval solutions that can help authors identify missing citations in their papers by identifying sentences that require citation, as well as recommending possible sources. This can be done with no additional human input besides the paper itself, making it easy to integrate in other applications.

By building upon existing works, models, and methods extracted from the literature, each step of the pipeline brings improvements leading to better overall performance or usability. In short, our solution can:

- identify text that requires citation with a **10 point F1-score improvement** over the current state-of-the-art model [10] by leveraging a fine-tuned large language model
- recommend papers to cite with a **0.5 Recall@5** while maintaining flexibility regarding the set of papers that can be recommended using semantic similarity search and re-ranking
- achieve high quality recommendations by combining two machine learning tasks that are traditionally studied separately (citing sentence classification [11] and citation recommendation [9])

Therefore, our pipeline is a high performance all-in-one solution for missing reference identification which has the potential to help researchers write higher quality papers by detecting their citation related mistakes automatically and offering insights that can help them alleviate the issues.

1.6 Paper Structure

Moving forward, we will deeply evaluate the need for our work in chapter 2, *Motivation*, exploring the different ways in which scientists fail to use citations correctly.

In chapter 3, *State of the Art*, we'll investigate existing models and methods that solve tasks adjacent to our work. More specifically, we'll explore literature related to citing sentence detection [11] and citation recommendation [9].

Then, chapter 4, *Experiments*, will touch on various failed experiments that did not end up being directly part of our final solution. This chapter will clarify the chains of thought that led us to the most influential ideas behind the other, more successful, experiments.

Chapter 5, *Proposed Solution*, will offer a high level overview of our architecture and explain each step of our solution, while chapter 6, *Implementation Details*, will dive deeper into the technologies, code and methods that are used to power the system.

Finally, chapter 7, *Evaluation*, will deeply explore the performance of our solution by different metrics, as well as compare it to existing solutions.

2 MOTIVATION

Learning how to properly cite other works is a complicated process that is easy to get wrong. This is especially true for junior researchers or students, who are often more focused on delivering their contribution, as opposed to presenting their work in the scientific context in which it was developed, leading to hard to decipher arguments.

A common problem is authors considering popular and established concepts to be familiar to all readers. While this is a valid opinion, it is definitely not an excuse for not citing the works that introduced those concepts in the first place. This assumption of over-familiarity damages the integrity of the paper and the quality of the global citation graph.

Publications whose findings are now taught in university level courses are particularly vulnerable to this, as authors might have a deep understanding of the subject without having ever read the original paper. For example, perceptrons [12] are a core machine learning concept that a lot of people learn about from introductory undergraduate-level courses^{1,2} without being exposed to the source paper, making it likely that they'll take them for granted in their explanations.

Furthermore, named entities like websites, libraries, datasets, models etc. may or may not have associated papers which need to be cited. For instance: the HuggingFace Transformers library has an associated paper [13], but the most popular HTTP requests library³ for Python⁴ does not. Research by Lin et al. [6] has shown that a large number of papers make this mistake, with some not even including links as a footnote.

Of course, there are also simple human errors. Authors can just forget to correctly mark their citations even if they are perfectly aware of the works that they sourced from. At a large scale, mistakes are inevitable [14] due to limited human performance, errors are therefore expected no matter how banal they are, especially considering the recent increasing rate of academic research [1].

As we can see, a lot of the mistakes that scientists make are pretty basic, coming from a lack of knowledge of the literature or human error. It is our hypothesis that these two shortcomings can be mitigated using computer assistance in the form of knowledge retrieval systems [7] and specialized machine learning models, offering a valuable tool for researchers. Of course, it's likely that very complicated cases of missing citations that require advanced reasoning exist, but the goal of this paper will be to make sure that the aforementioned common patterns will

¹<https://openlearninglibrary.mit.edu/courses/course-v1:MITx+6.036+1T2019/course/> accessed on 30th April 2025

²<https://cs229.stanford.edu/> accessed on 30th April 2025

³<https://requests.readthedocs.io/en/latest/>

⁴<https://www.python.org/>

be resolved.

Taking all arguments into account, the study of methods for detecting missing citations solves a harmful problem that poisons the academic community and can help researchers write better publications, save time during peer review, and correctly credit their peers.

3 STATE OF THE ART

Understanding the nature of citations has become an important sub-area of research on scholarly documents through tasks such as citation intent classification [15] or named entity recognition [16]. These developments are somewhat fueled by the rise of models and systems that use citations as an input to build models that understand the relations between documents [17], improve retrieval systems [18], or analyze the evolution of an entire field of study [19].

Our end goal of automatically detecting passages where citation is needed and recommending papers to back up those passages is most similar to the following two tasks:

1. **Citing Sentence Classification** (also known as **Cite-Worthiness Detection**), introduced by Sugiyama et al. [11] (and later re-coined by Wright et al. [10]), is the task of classifying sentences as either citing or non-citing, meaning whether a passage contains references to other works or not. This is done without the aid of reference markers (e.g: [1], [1, 2, 3], [1-5], Dumitrache et al. etc.).
2. **Citation Recommendation** is the task of recommending papers to cite based on a target paper. It is often categorized by the context used for recommendation:
 - (a) **Global Citation Recommendation**, introduced by Strohman et al. [20], uses only the title and abstract of a paper (known as the global context) to recommend references to add to the bibliography.
 - (b) **Local Citation Recommendation**, introduced by He et al. [9], is the task of recommending citations based on the location of citation marker placeholder, as well as its surrounding text (called a local context). This method sometimes uses the global context as an extra input.

In the following sub-sections, we will be discussing and evaluating each of the previously discussed concepts in detail, identify shortcomings, as well as propose ideas for improvement and paths to follow in our experimentation.

Additionally, we will explore a novel method of citation recommendation specifically designed for scientific entities that leverages a statistical approach, stepping out of the machine learning realm that most existing solutions reside in.

3.1 Citing Sentence Classification

The task of citing sentence classification has only been studied as a supervised machine learning problem, as in taking a model architecture and training it on sentences labeled as citing or non-citing. Datasets for this task are mostly generated from large corpora of scholarly text labeled automatically using existing reference markers, which are then removed.

There are three main publications tackling this problem, each of them making subsequent improvements related to the model architecture used, as well as the quality of the dataset. We will discuss these in chronological order.

3.1.1 Support Vector Machines and a Basic Corpus

Sugiyama et al. [11] used SVMs (Support Vector Machines) [21] on a dataset built on the ACL Anthology Reference Corpus [22]. To label sentences, they used regular expressions to detect reference markers and removed them afterwards without any extra heuristics.

Unfortunately, the exact methods used to generate their dataset were not provided, but we do know that it is highly imbalanced (approximately 90/10 split between non-citing and citing sentences) and likely has problems such as sentences losing their correctness after removing reference markers. The authors also fail to acknowledge that accuracy is an irrelevant metric given the imbalance in labels (e.g: 0.9 accuracy can be achieved by simply marking everything as non-citing), no mentions of other metrics such as precision, recall or F1-score exist.

Therefore, while their work is an important stepping stone, it will not be included in further comparisons, mostly due to lack of support in reproducing their results and the fact that further work [23] proves that SVMs [21] are outpaced by more modern alternatives for this task.

3.1.2 Recurrent Convolutional Neural Networks with an Extended Corpus

Färber et al. [23] leveraged a recurrent convolutional neural network architecture and an extended corpus of scholarly documents compared to previous work to improve results by all relevant metrics.

Their corpus [24] is an extension of the previously discussed one. Methods for reproducing the labeled dataset were not provided, but the description offered leads us to believe only detection and removal was performed. It suffers from the same problems of class imbalance, but uses over- and under- sampling to improve results in both their baseline SVM and RCNN model.

In the end, its results vastly outpace SVM based solutions with a 0.5 F1-score for the RCNN

approach, compared to a range of 0.03-0.15 F1-score among all SVM variants.

3.1.3 Transformers, Paragraph-Level Context and an Improved Method for Labeling Scholarly Text

Wright et al. [10] propose CiteWorth, a high quality dataset based on the extremely large (around 80 million papers) S2ORC [25] corpus, leveraging its high quality and detailed citation contexts. To ensure that the dataset maintains a high standard of quality, in addition to detecting reference markers and removing them, it also employs heuristics to:

1. Ignore samples where removing reference markers would break grammar, e.g: "As shown by [1, 2, 3]" is an incomplete sentence without the reference markers and, thus, it would be ignored.
2. Ignore samples that are detected as having reference markers despite S2ORC not marking them as such.
3. Only include sentences that are at least 20 characters long.
4. Only include sentences that follow basic conventions (capital letter, ended with normal punctuation etc.).

They also employ manual human evaluation to compare these heuristics to a baseline detect-and-remove method, proving it leads to a higher quality dataset.

CiteWorth [10] also groups all sentences by paragraph and only takes into consideration paragraphs whose sentences respect the aforementioned rules. This is done to easily incorporate the larger context of a sentence in its classification. An intuition for why this could be helpful can be developed through the following paragraph extracted from the SciBERT paper [26]:

The BERT model architecture (Devlin et al., 2019) is based on a multilayer bidirectional Transformer (Vaswani et al., 2017). Instead of the traditional left-to-right language modeling objective, BERT is trained on two tasks: predicting randomly masked tokens and predicting whether two sentences follow each other. SCIBERT follows the same architecture as BERT but is instead pretrained on scientific text.

As we can see, the first mention of BERT [27] in the paragraph cites the paper that introduced the model, but later mentions do not. This is fairly common behavior for most authors, but can be misleading when training a model at the sentence level.

Next, the authors fine-tune various pre-trained Transformer [28] models, with the most successful being SciBERT [26] for sentence level classification and Longformer [29] for classifying sentences in the context of their paragraphs.

The resulting models have better metrics than the previous ones, at 0.62 F1-score for single-sentence classification and 0.67 for multi-sentence/paragraph-level classification.

3.1.4 Comparison and Moving Forward

As seen in Table 1, the Longformer-based [29] model outperforms all other options, making it the current documented best model for this specific task. It is also a model that is fairly reasonable to run on local, consumer-grade hardware, making it perfect for integration into various tools.

Model	Precision	Recall	F1-score
RCNN	49.57	65.56	56.41
SciBERT	57.03	68.08	62.06
Longformer	59.92	77.15	67.45

Table 1: Performance metrics for different SOTA models on citing sentence classification using the CiteWorth dataset, all weighted

This leads us to the hypothesis that we could improve performance even more by looking at sentences in a larger context or, at least, using full attention instead of sparse attention at the paragraph level. Since the release of the papers discussed so far, we have seen the rise of large language models with larger and larger context windows [30, 31], leading us to ask: what if we used them as citing sentence classifiers? This idea will be explored in further chapters.

3.2 Citation Recommendation

The task of citation recommendation involves offering a list of candidate papers to cite based on a target paper as input. It has been approached from multiple angles, but many of the existing approaches are not a good fit for our use case.

3.2.1 Using Global Context

The global citation recommendation [20] task boils down to recommending papers to include in the bibliography based solely on the title and abstract of the target paper. While this mostly works if we are simply interested in seeing what the most similar papers look like, it is a way less focused approach that doesn't give enough information regarding why a paper should be cited.

Since we are interested in finding missing citations based on actual statements made in the

content of the paper that users can assess, we can safely ignore this type of citation recommendation.

3.2.2 Using Local Context

The local citation recommendation [9] task uses a specific context (some text) with a placeholder citation marker to recommend citations. This placeholder mostly stems from the fact that these systems only focus on recommending citations, often considering the previously discussed task of citing sentence detection as solved or proposing users to input the model with sentences manually. Our work will do without them.

Actual implementations vary in approach, using models such as topic modeling [9] or neural networks [32, 33]. While these approaches exhibit fairly decent results, they unfortunately have scalability issues when it comes to extending the number of papers that can be recommended by the model, requiring re-training whenever new publications have to be added.

This makes these models quite unwieldy for real-life scenarios as they constantly require compute effort to re-train due to new papers coming out at an increasing rate [1]. Due to these issues, we will pursue the design of a new local citation recommendation model.

3.2.3 Next Steps

Local citation recommendation mostly describes what we're looking for, but current approaches are not viable given the objectives we defined in section 1.3, meaning that current solutions aren't flexible enough to support an evolving set of papers.

Since this is a very knowledge-intensive task, we propose exploring retrieval-augmented generation [7] style methods for retrieving papers relevant to a target context. This type of retrieval system only requires indexing new papers through text embeddings, solving our scalability problem. This will be explored in more detail in the following sections.

3.3 Scientific Entity Recognition and Recommendation

While most solution for detecting missing citations and recommending references rely on sub-symbolic representations of the problem to achieve their results, Lin et al. [6] propose a natural language processing method based on the detection of scientific entities, a special case of named entity recognition [34] that focuses on detecting terms that represent concepts published in scientific papers.

This method uses the number of co-occurrences between recognized scientific entities and citations in the same sentence to map concepts to their source papers. These mappings are generated statistically by computing co-occurrences on the S2ORC dataset [25], a large

corpus of scientific papers, and can then be used for citing sentence detection and citation recommendation on target papers by marking passages that contain scientific entities as needing to cite the associated paper.

While the proposed solution manages to solve two separate tasks simultaneously: citing sentence detection and citation recommendation, it only does so for the very specific case of missing citations of scientific entities which, as discussed in chapter 2, does not represent all cases of missing references.

This issue is also reflected by the fact that the method does not have an evaluation done on any relevant datasets, the main form of evaluation being performed by applying it on papers to discover missing citations. Therefore, with no clear path to generalizing this method to more broad use cases due to the complexity of the problem, we will focus on sub-symbolic approaches in our solution.

However, this is not to say that symbolic approaches do not have their place in this problem. Arguably, their high explainability could lead to easier to understand responses by offering the reason why a passage requires citation (uncited scientific entities, statistics/percentages without a source etc.).

4 EXPERIMENTS

Though they are not necessary for understanding the final proposed solution, the failed experiments outlined in this chapter present the logical set of steps that were taken in our research, each one offering us a new direction to explore. Therefore, they are important for gathering insights and developing a deep understanding of our problem.

4.1 Large Language Models for Identifying Missing Citations

LLMs have proven to be adequate for a large set of machine learning tasks without any targeted fine-tuning of the model besides a couple of example inputs/outputs [35], making them an even more convenient solution than popular transfer learning [36] methods. It is therefore worth our time to explore whether our problem can simply be solved through prompt engineering before implementing a specialized solution.

4.1.1 General Approach

We used the popular ChatGPT interface to evaluate GPT4 [30] for missing reference detection by asking it to reconstruct the bibliography of a given paper using real scientific works and backing up its choices with quotes from the paper, covering our needs of classification and retrieval described in previous sections.

We evaluated two different scenarios: a realistic version where the target paper does not have citation markers (e.g: [1], [2, 3] etc.) and a version with them included. The model was also tested on papers with very different characteristics.

The problem was described in natural language using the following prompt:

You are a highly experienced scholar that constantly reads scientific papers. I have this scientific paper text from a conference, but I lost all its references. Could you take a look and attempt to find papers that are referenced by this paper? I'm interested in the name of the paper, its authors and the reason why you think so (preferably quoting the original text). I'm only interested in papers you are 100% sure exist.

Please keep in mind that this paper is from a renowned scientific conference, meaning it is likely that all of its references come from academic works such as scientific journals and scientific conferences.

I will now proceed to give you parts of the text in separate messages, as it's too long for one message. After each part, please give me information about the papers you think are cited. Keep in mind that not all excerpts I send you need to contain references, please only tell me about papers you're sure are referenced.

Our prompt is designed to describe the problem, its input, and its output, as well as reinforce the idea that we want real sources. A lot of our experiments suffered from being recommended low quality sources, usually offered by web search agentic tools integrated into the chatbot. To fix this, additional instructions are given to force the model to only consider academic works as candidates in order to avoid referencing informal articles and other unreliable sources.

The prompt features slight modifications in the scenario where citations markers are still present, adding the following sentence to the problem description: *Meaning, for each "[x]" or "[x, y]" or "[x-x+n]" mark, try to figure out what papers were being cited..*

4.1.2 Testing

We tested both scenarios on three separate papers with various characteristics:

1. A paper [37] still in peer review at test time¹ which cites a lot of cutting edge research
2. An older, very popular paper [28] ($\sim 100k$ citations)
3. An older, less popular paper [38] (~ 50 citations)

We chose these papers strategically so we can evaluate the model's performance on papers it has surely been trained on (such as paper 2), as well as on papers that it hasn't been trained on or whose findings weren't reinforced through external sources (such as paper 1 and paper 3, respectively). A comparison between the performance of the model on each paper and scenario combination can be seen in Figure 1.

Clearly, the performance is quite poor overall. Besides the weak metrics, some other issues that have come up are:

1. The total number of recommendations given for scenarios with no citation markers is fairly low compared to the real number of references.
2. Some recommendations are hallucinated [39] papers that do not exist.
3. Sometimes when a correct title is offered, the wrong authors are mentioned.
4. It is mostly unable to recommend any paper that isn't popular and fairly old. This is likely because newer, niche papers weren't used in the training set.
5. The model often recommends the same paper as a candidate for multiple reference markers.

¹Early January, 2025

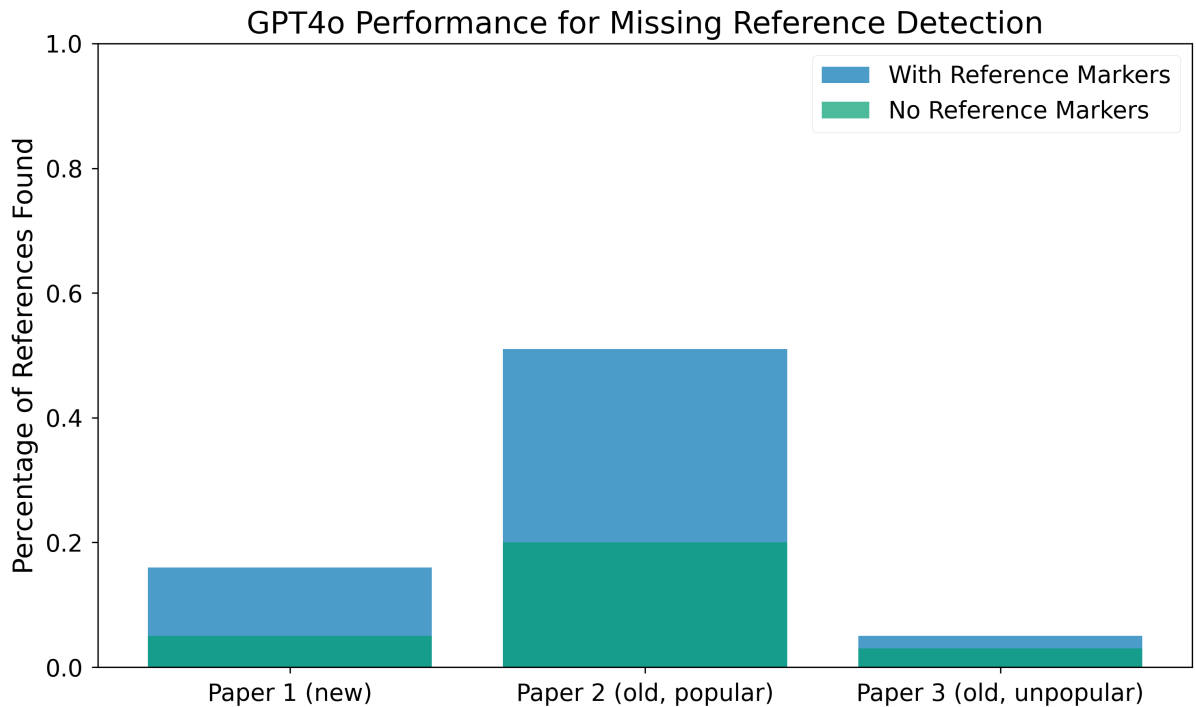


Figure 1: Bar plot comparing performance of GPT4o for missing reference identification by the percentage of references identified

4.1.3 Conclusions and Next Steps

These observations are enough to rule out the possibility that a pre-trained LLM could solve our problem without any specialized solutions (at least currently). The poor performance on a new paper and the lack of recently published candidate papers shows that our main motivation, verifying citation integrity before publishing, is not satisfied.

We can conclude that it is worth pursuing a specialized solution to our problem. Based on the issues we identified earlier, our solution will definitely need a way to ground its recommendations to a real and varied set of papers, as well as be able to detect passages that need citations with a higher accuracy.

4.2 Citation Recommendation Using Text Embeddings

Information retrieval systems that leverage text embedding models have become popular due to their flexibility and ease of use. These models can encode text in an n -dimensional vector space, placing semantically similar sequences of text close to each other. This allows us to use metrics such as dot-product or cosine similarity to determine how semantically close two sequences of text are.

A typical architecture for such a system often involves a database of vectors that represent documents. Searches can be performed on the database by generating an embedding of

the query and performing a k nearest neighbor search. This is often the basis of retrieval augmented generation systems [7].

4.2.1 Testing Scenario

We experimented with a couple of text embedding models for retrieving paper abstracts based on a given text sequence. Our experimental setup uses a database of around 50k paper abstracts randomly extracted from S2ORC [25] and a couple of sample passages which should cite a certain paper (which is also indexed in the database). This is done to see if the embedding model is capable of finding the correct paper among a larger pool of candidates, as opposed to just verifying the order against a couple of examples.

The passages we chose are fairly simple, often mentioning concepts which have an entire paper dedicated to them by name, an example being: *"We used the Adam optimizer."*. Our goal was to filter through models fast on the premise that we probably can't get decent performance out of a model that doesn't work well on even the simplest of cases.

4.2.2 Results

Unfortunately, testing this scenario with a couple of popular text embedding models such as all-MiniLM-L6-v2² and bge-m3 [40] did not produce satisfying results. None of the models managed to retrieve a single correct paper in their top 100 results, not justifying further evaluation.

This is likely due to the complexity of the task at hand, as well as the fact that the semantic characteristics of our queries and abstracts are not necessarily similar enough to warrant our high expectations. A sentence which cites a certain scientific paper does not have to be semantically similar to the abstract of the cited paper, it's more likely be closer in the vector space to other sentences which refer to the cited paper.

To alleviate this, we decided to experiment with instruction-tuned text embedding models. Their main advantage is that they allow us to describe our task in natural language, encoding text and a description of the task in a single vector. This idea ended up passing our preliminary tests, making it worth further evaluation. Our final solution now uses E5 Large Instruct [41], as described in chapter 5.

²<https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>

5 PROPOSED SOLUTION

Our solution consists of a multi-step pipeline that encompasses both citing sentence detection [11, 23, 10] to identify passages with missing citations, as well as a local citation recommendation [9] system powered by retrieval based on cosine similarity of text embeddings. Figure 2 describes the entire process for a single sentence.

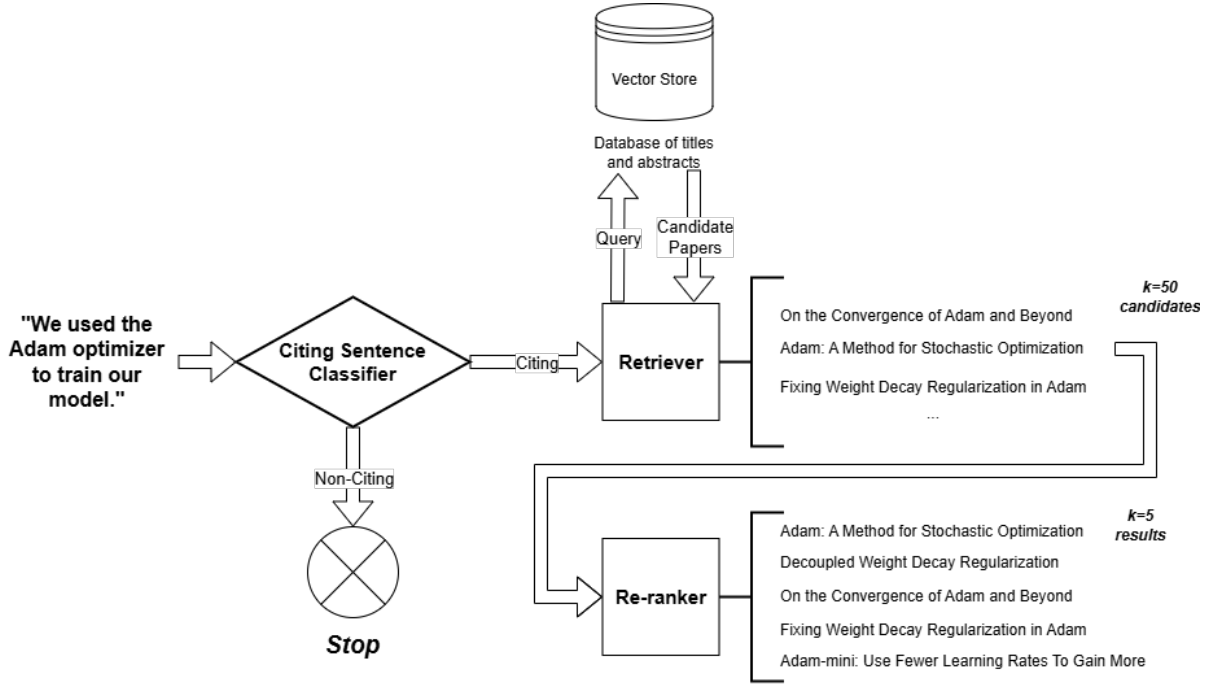


Figure 2: Steps for identifying missing citations for a single sentence.

A citing sentence classifier is used to decide whether a given sentence requires citation or not. Only sentences that need citations are passed to the retriever, which builds a query to our vector database to retrieve $k = 50$ candidate papers.

While the retriever is very fast and scales well to a large number of papers, it is not accurate. We deploy a second step to our retrieval process, which re-ranks the candidate papers using a large language model [42, 43]. This was done to balance between quality of results and time performance, offering us the best of both worlds: fast retrieval through vector search and quality ordering through LLM re-ranking.

To respect Miller’s Law [44], we only return the top 5 papers from the reordered list. This makes it easy for users to scan the proposed solutions quickly without getting overwhelmed.

This pipeline can then be applied to an entire paper as input, as seen in Figure 3, to offer a full report of missing citations across the document.

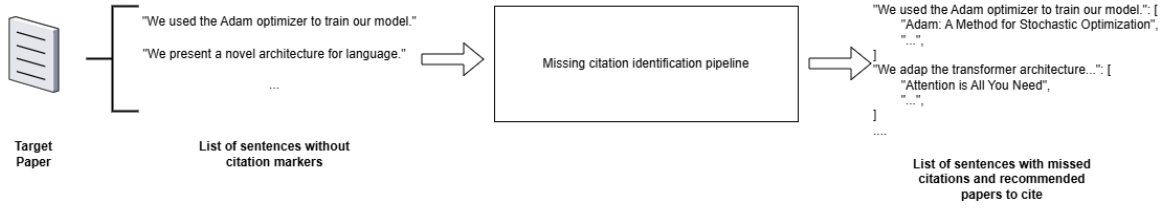


Figure 3: Identifying missing citations for an entire paper.

The output of our pipeline can be summarized as a list of sentences which should’ve used citations and associated lists of candidate papers to cite for each of those sentences.

The following sections will describe each step of our solution in detail.

5.1 Citing Sentence Detection

We use a cite-worthiness detection model [11] at the paragraph level to decide which sentences require citation recommendation. As described in subsection 3.1.3, considering the paragraph context of each sentence highly increases performance [10] compared to single sentence classification, likely due to varied citation practices among researchers such as only citing a paper once per paragraph.

Our classifier model is a version of Gemini Flash 2.0 [45] fine-tuned on the CiteWorth dataset [10] to classify sentences in the context of their paragraphs by generating a label for each sentence. We improve on the existing state-of-the-art, that being Longformer [29] trained on the same dataset, by tackling two of its main limitations:

- Longformer [29] uses a sparse attention mechanism [46] to handle longer passages, trading performance for a larger context window while maintaining almost the same number of parameters as models with a smaller window and full attention (such as RoBERTa [47])
- it has a parameter count of around 100 million [29], a fairly low number compared to newer language models

This led us to the hypothesis that improved performance could be achieved using a language model with full attention, a large enough context window to fit a regular paragraph, as well as more parameters. LLM families such as GPT [30] and Gemini [31] satisfy all of these requirements.

Even though training or fine-tuning such a model used to be out of reach for most users due to hardware constraints, services for this kind of compute have become a convenient offering

of cloud platforms like Vertex AI, making it a viable option even without access to a cluster or specialized hardware.

Taking advantage of this, we fine-tuned every parameter of Gemini Flash 2.0 [45] in a supervised manner by training it on whole paragraphs as input and citing/non-citing labels for each sentence in each paragraph as outputs to compare to. We also add additional formatting to the inputs and reference outputs in the form of sentence number tags to make it easier for the model to count, a problem that LLMs often have a hard time solving [48].

Details regarding the dataset used for fine-tuning, CiteWorth [10], can be seen in Table 2

Metric	Value
Sentence count	1,181,793
% citing	31.76%
% non-citing	68.24%
Train/Test/Validation split	80/10/10
Train sentences	945,426
Validation sentences	118,182
Test sentences	118,185
Scientific field coverage	10 different fields
Publication years	Up to 2021

Table 2: Statistics regarding the CiteWorth dataset [10]

Our inputs are therefore formatted as follows:

0: <sentence>

1: <sentence>

...

n: <sentence>

These inputs also have associated labels which can be used to update the parameters, tuning the model:

0: non-check-worthy/check-worthy

1: non-check-worthy/check-worthy

...

n: non-check-worthy/check-worthy

This setup has allowed us to fine-tune the model into a generative cite-worthiness classifier that beats out the performance of the previous state-of-the-art model by all relevant metrics on the test split of CiteWorth [10] (over 100k sentences), gaining over 10 F1 points over it. An in-depth comparison can be found in section 7.1.

Of course, running this model locally on most consumer-grade systems is not realistic due to its complexity, but inference in a cloud environment is fast and fairly inexpensive¹, mostly due to the low number of output tokens required. It is likely that this will become even more accessible in the future, either due to capable hardware being more generally available or similar advances in cloud offerings.

5.2 Retrieval

We use a database that stores paper titles and abstracts as vectors generated by a text embedding model, E5 Large Instruct [41]. This model allows us to place queries (citing sentences) and relevant paper text close to each other in the vector space, allowing us to retrieve candidate papers based on the distance between vectors.

Being an instruct embedding model, we can also use natural language to describe the retrieval task at hand before offering it our query sentence, conditioning the generated embeddings on a specific task. The following description of our task is used to encode the queries:

Given a sentence where a paper is cited, find the abstract of the paper it cites.

Evaluating this retrieval method on the PeerRead dataset [49], a large corpus of citing sentences/contexts and papers to recommend from, results in a Recall@50 of 0.46. This means that for 46% of citing sentences provided by the dataset, the correct paper that should be cited appears in the $k = 50$ candidate papers.

A couple examples of retrieval results can be seen in Table 3, showcasing some scenarios that the retriever excels at, finding scientific named entities. However, the method is sometimes limited by its usage of semantic similarity, the fourth example in Table 3 is a scenario in which the usage of the term *Berkeley-Parser* serves as more of a hindrance since the original paper did not coin this term for their innovation, it came to be after the paper was published.

¹Less than 1 US Dollar for inference on the entire CiteWorth test split of over 100k sentences, tested through Vertex AI

Sentence	Cited Paper	Position In Results
<i>We used the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$.</i>	Adam: A Method for Stochastic Optimization	2
<i>... we replace our sinusoidal positional encoding with learned positional embeddings...</i>	Convolutional Sequence to Sequence Learning	4
<i>Recent work has achieved significant improvements in computational efficiency through factorization tricks...</i>	Factorization tricks for LSTM networks	3
<i>...the Transformer outperforms the Berkeley-Parser.</i>	Learning accurate, compact, and interpretable tree annotation	Not found

Table 3: Retrieval results on some sample sentences extracted from Vaswani et al.[28], retrieved from a database of around $50k$ papers which includes each cited paper.

To get the closest vectors to our query vector, the database system uses a fast approximate k-nearest-neighbor algorithm to determine $k = 50$ similar vectors to our query by dot-product², this allows us to comb through a large databases of embeddings and get relevant candidates without scaling linearly (an exact KNN search is $O(n)$).

Our model choice is based on benchmark results³, where E5 Large Instruct [41] ranks very well. The only models that outperform it are unreasonable to use on local systems, as they can have over 14 times more parameters than E5. For reference, such a model takes around a minute to generate a single vector on a consumer GPU.

While this method does give us a relevant list fairly quickly, its order is rarely accurate even for obvious scientific named entities, as seen in Table 3. The main purpose of this step is to fetch a list of likely candidates from a large database of papers in a short time. Our next step is, based on the origin of each returned vector, to construct a list of papers (title + abstract) of candidate papers that can be passed to a re-ranking step.

5.3 Re-ranking

To improve the order of our candidate papers, we employ a large language model based re-ranking method in the form of RankGPT, introduced by Sun et al. [42, 43]. RankGPT uses

²Arguably, cosine similarity is used, but because we normalize our vectors, it becomes equivalent to the dot-product of the vectors.

³<https://huggingface.co/spaces/mteb/leaderboard>, intfloat/multilingual-e5-large-instruct is 4th place, last accessed on 8th May 2025

LLMs to rank text passages according to a given prompt by leveraging a listwise-approach for instructional permutation generation, basically using LLMs as comparison functions between passages.

The passages we will re-rank are simply the title and abstract of each paper concatenated. They will be re-ordered according to the same prompt we used for the embedding model along with the citing sentence we are targeting. This method produces quality results, almost always improving the initial list, as can be seen in Table 4.

Even though RankGPT [42, 43] could be theoretically deployed as a pure retriever that outputs higher quality results without a two step process, it does not scale well to a large number of papers (even re-ranking 50 papers takes longer than the retrieval step), making it too slow to deploy at the scale of an entire database of papers, but perfect as a re-ranker.

Sentence	Cited Paper	Initial Result	After Re-Rank
<i>We used the Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.98$ and $\epsilon = 10^{-9}$.</i>	Adam: A Method for Stochastic Optimization	2	1
<i>...we replace our sinusoidal positional encoding with learned positional embeddings...</i>	Convolutional Sequence to Sequence Learning	4	2
<i>Recent work has achieved significant improvements in computational efficiency through factorization tricks...</i>	Factorization tricks for LSTM networks	3	1
<i>...the Transformer outperforms the Berkeley-Parser.</i>	Learning accurate, compact, and interpretable tree annotation	Not found	Not Found

Table 4: Re-ranking results on some sample sentences extracted from Vaswani et al.[28], retrieved from a database of around 50k papers which includes each cited paper. Only top 5 taken into account in re-ranking results.

The preliminary results of Table 4 are also backed up by a large scale evaluation performed on the PeerRead dataset [49]. We used our retriever to generate lists of candidates and passed them to the re-ranker. To evaluate the re-ranker fairly, we supplement the lists with the correct paper if it’s not already present. This is done to prevent an analysis of the re-ranker that caps its performance to the retriever. To better illustrate this issue: if the correct paper is present only in 40% of lists, then re-rank evaluation on the other 60% is in vain, as we have no other means of evaluating quality besides the position of the cited paper.

The final result is a Recall@5 of 0.49 if we force the correct paper to be in the initial list, showing the independent quality of the re-ranker.

Faster, more traditional re-ranking models such as bge-reranker-base [50] were not able to match its performance. In fact, any model that did not feature the ability to describe the task at hand in natural language has failed, even offering worse results than our retriever.

6 IMPLEMENTATION DETAILS

This chapter will zoom in on our solution to discuss the technical details that make it a good candidate for further integration into existing applications. We will touch on the provided interface, the technologies and frameworks used, small code details that improve its quality, as well as important design decisions that were taken.

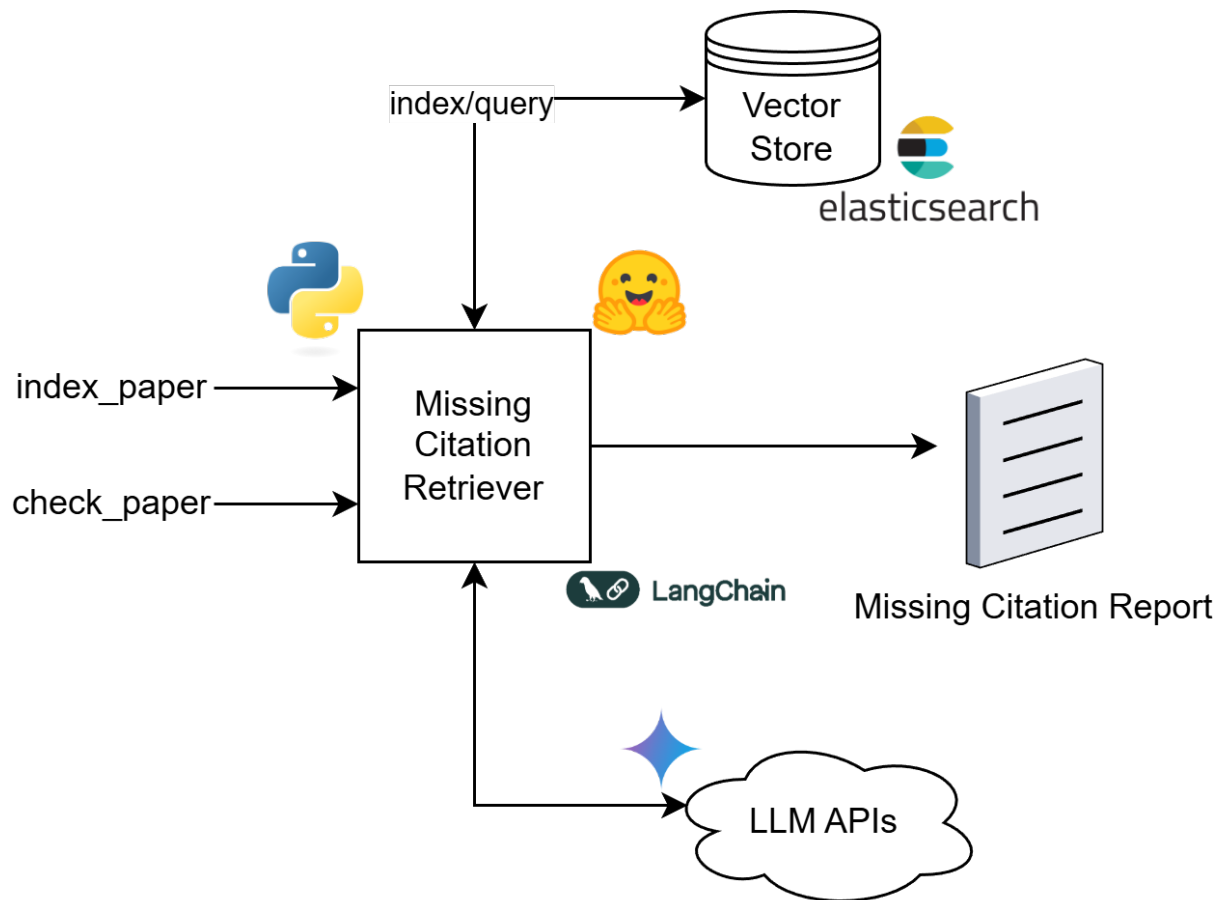


Figure 4: Visual representation of our solution's architecture

The culmination of all our experiments, the all-in-one pipeline described in Figure 2 and Figure 3, is represented by two main entities:

- a module called *missing_citation_retriever* that exposes methods for indexing new papers in the database and running papers through the missing citation detection pipeline
- an external vector database that is used to store embeddings, accessed by the module when querying for candidates

A visual representation of this setup can be seen in Figure 4. The module is written in Python, taking advantage of its vast ecosystem of high performance libraries for developing machine learning applications and ease of use. It handles processing papers that need to be indexed or verified for missing citations, communicates with the database when paper candidates are needed and handles LLM calls. The following sections will describe how we approached the implementation.

6.1 Orchestration

LLM app development is evolving at a break-neck pace with new models from different providers coming out constantly [51, 30, 45]. This makes it difficult to make correct design choices considering the fact that tomorrow's state-of-the-art model might be coming from a new company and have a radically different API. Therefore, due to our usage of large language models and text embeddings, we decided to organize our implementation using the LangChain framework¹ for Python.

The purpose of the framework is to offer a fully featured solution for orchestrating LLM apps, providing interfaces to access different AI providers, databases, process documents etc. that are unified and easily interchangeable. This approach helps mitigate potential vendor lock-in and tech debt, allowing us to adapt as needed.

For instance, there is no need to translate between LLM APIs (Application Programmable Interfaces) if we decide to switch AI providers, all AI services are accessed through the same interface in LangChain. Similar features are present for vector databases and embedding models, allowing us to switch between them easily.

LangChain also helps organize workflows through its state graph feature, allowing us to model the different stages of our pipeline nodes which transmit information between them through their internal state.

6.2 Embeddings

To generate embeddings of paper titles and abstracts, we use the HuggingFace transformers library [13] directly through the unified LangChain interface. This allows us to easily access open-weight models such as E5 Large Instruct [41].

Most paper abstracts are way too long to be encoded as a single embedding since the similarity score between our query and the entire abstract compared to the score between our query and smaller passages can be quite different. Even if this wasn't the case, most embedding models limit the number of tokens they accept as input due to performance issues, as the self-attention mechanism used by the transformer architecture scales $O(n^2)$ to the length of

¹<https://python.langchain.com/>

the input in terms of memory and compute [52]. For instance, our model has a cap of 512 tokens, which almost certainly won't fit any abstracts.

This is why we need to split our text before generating embeddings. We concatenate the title and abstract for each paper that we want to index and split at 300 characters with an overlap of 100 characters between chunks to minimize the number of nonsensical passages. This provides a balance between the number of embeddings generated and the specificity of their content.

All embeddings are also normalized before being stored in the database, removing the magnitude/confidence of our vectors. This is the way our current model choice is recommended to be used and also serves as a small optimization that saves some compute at inference time since cosine similarity on normalized vectors is equivalent to dot-product, an operation that is easier to compute. A proof of this can be seen in Equation 1, it assumes that vectors A and B are already normalized, therefore their lengths are 1.

$$\text{cosine_similarity}(A, B) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|} = \frac{\mathbf{A} \cdot \mathbf{B}}{\mathbf{1} * \mathbf{1}} = \text{dot_product}(A, B) \quad (1)$$

6.3 Vector Store

We opted for Elasticsearch² as a vector database because:

1. it has a favorable open-source policy compared to a lot of proprietary solutions (Pinecone, Weaviate etc.).
2. it allows storage of objects besides vectors, such as plain text and metadata, allowing us to use a single database instead of two.
3. we can supplement our vector searches with filtering rules.
4. it is way more flexible and easier to use than something like PostgreSQL + pgvector, being a NoSQL solution.

Our database contains two indexes: one for plain text entries of papers and one for storing embeddings (and linking back to the plain text entries). This allows us to easily retrieve the title and abstract of a paper represented by a certain embedding without doing much computation during inference.

Of course, Elasticsearch can be easily replaced by any other vector database by altering a couple lines of code thanks to LangChain.

²<https://www.elastic.co/enterprise-search>

6.4 LLM APIs

Classifying sentences or re-ranking using API calls to large language models can be fairly slow, especially if we perform the calls sequentially by waiting for each individual call to finish before starting another one. To speed this process up, we employ concurrency through usage of a thread pool which waits for API calls to finish asynchronously, improving our IO-bound performance.

Chatbots, the classic LLM usage scenario, require some degree of randomness in their responses to avoid having deterministic outputs, allowing interaction with them to feel natural and less robotic. This randomness is provided by the temperature parameter, whose higher values offer the model a chance to select tokens besides the number one most likely next token. However, randomness is not at all beneficial to our classifier model since we want completely deterministic classifications. We will therefore set the temperature to 0 in our LLM calls.

Even though LangChain allows us to easily switch between AI providers for re-ranking if needed, the user might want to use different providers due to personal preference. The module offers the option to choose your provider through a strategy pattern, the current options being OpenAI and Gemini. This pattern also makes it easy to integrate other services in the future if desired.

7 EVALUATION

This chapter will provide a quantitative analysis of each step of our pipeline, comparing our solutions to the current state-of-the-art while discussing their respective advantages and drawbacks. To determine if our pipeline is a reasonable solution in terms of speed, we will evaluate its execution time across multiple input papers. Additionally, we will also discuss potential directions for future research based on these findings, highlighting ways in which our solution could be improved even further.

7.1 Citing Sentence Detection

As discussed in section 5.1, the dataset we used for training and testing is CiteWorth [10]. This is mostly due to it being the only dataset that attempts to improve the labeling process for this task through various heuristics, while also offering quantifiable metrics such as manual human evaluation compared to a baseline to determine its quality. All other comparable datasets [23, 11] rely exclusively on automated labeling based on the presence of citation markers, a method which often leads to broken grammar and other features of low quality training examples.

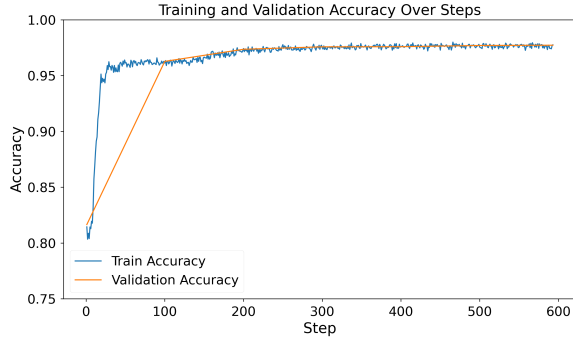
Analyzing the accuracy and loss of the fine-tuned Gemini Flash 2.0 model during training, as seen in Figure 5¹, reveals that the model almost reaches its maximum performance on the validation set after a fairly low number of training steps (around 20), highlighting the power of transfer learning through fine-tuning large foundation models compared to classically training an architecture from scratch.

Based on this observation, we can determine that the amount of data used for training was more than adequate since no significant gains were made after 200 steps of training. This means that we won't be able to improve on our solution by simply acquiring more data to train on. Therefore, if we want to get better performance from our model, further research should look toward other means of improvement.

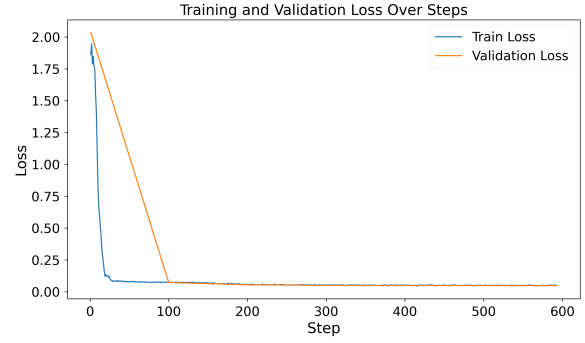
Using the test split of CiteWorth [10] of over 100k sentences, we performed a large scale evaluation of the fine-tuned model using accuracy, precision, recall and F1-Score as metrics [53]. The results of the evaluation can be analyzed in Table 5 and more easily visualized through Figure 6.

The most important metrics here are precision and recall (and implicitly F1-Score) due to

¹The metrics presented are based on next token prediction due to limitations of the external fine-tuning service we used.



(a) The model's accuracy between training steps on train and validation samples



(b) The model's loss between training steps on train and validation samples

Figure 5: Training and validation performance of the model over time.

Metric	Gemini Flash 2.0	Longformer	SciBERT	RCNN
Precision	0.78	0.60	0.57	0.49
Recall	0.76	0.77	0.68	0.65
F1-Score	0.77	0.67	0.62	0.56

Table 5: Evaluation of various citing sentence detection models on the test split of CiteWorth. SciBERT and RCNN performance was extracted from Wright et al. [10]

a slight class imbalance of 60% non-citing sentences to 40% citing sentences. Our model performs extremely well on these metrics, showing a 10 point F1-Score improvement over the next best model, Longformer. These findings prove our hypothesis that a full attention over a paragraph and a more complex model can highly improve performance on this task.

One of the drawbacks of our generative approach as opposed to other existing solutions is the possibility of outputs that do not respect the format used in the training data. For instance, there is nothing architecture-wise to force the model to provide labels for every single sentence in the input, meaning that it can very well decide to provide 3 labels for a 4 sentence paragraph. However, at least at our current scale, this does not seem to be a problem since the issue only popped up in one paragraph out of around 21k.

In our opinion, further research on this topic should approach the problem from a new perspective in terms of training input to achieve considerably better performance. A couple directions that could be explored:

- expanding the attention mechanism to entire sections of papers as opposed to just paragraphs
- leveraging additional paper metadata (domain of study, journal etc.) as input
- exploiting the citation graph by taking already existing citations into account

Additionally, we think there is potential to match this performance with a less complex (and

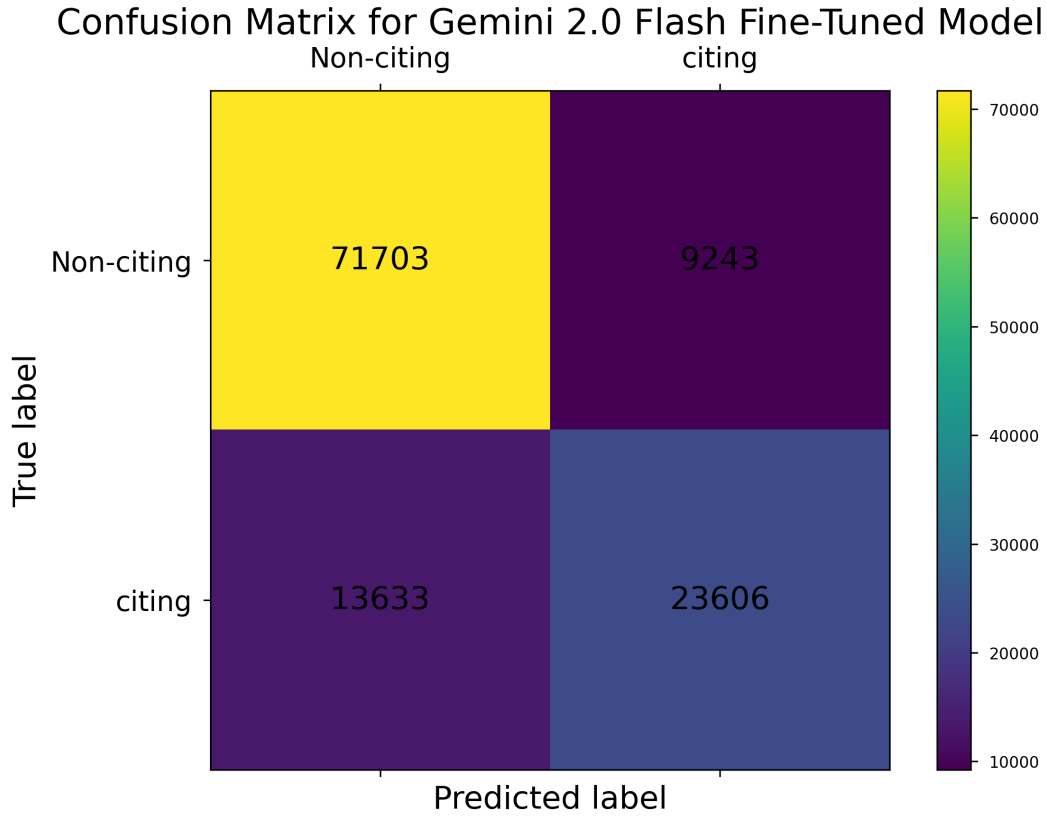


Figure 6: Visualization of the predictions of our fine-tuned Gemini Flash 2.0 model on the test split of CiteWorth [10]

therefore local-friendly) large language model such as the 8 billion parameter model of the Llama 3 family [54] as our current approach might be overkill for the given task. Alternatively, our model can be swapped out for the Longformer if local inference and speed are desired over additional performance.

7.2 Retrieval

We evaluated our citation candidate retriever on the PeerRead dataset [49]. This corpus features a set of papers that can be recommended and various passages (formally known as local citation contexts) which require citation recommendation. One advantage of this dataset is that it was sourced from peer reviews, a fairly realistic scenario for testing such a citation recommendation system. Details regarding this dataset can be observed in Table 6.

We measured our retriever’s performance using the Recall@K metric, meaning how often the correct paper appears in the top K results. This allows us to evaluate the performance of our retrieval model for ranking relevant documents from fixed set of candidates.

We will compare our results to Jeong et al.’s [8] and Yang et al.’s [55] neural network approaches on the same dataset to see how our method holds up to more potent, but less flexible models.

Metric	Value
Paper count	4,898
Citation context count	17,247
Publication years	2007 - 2017

Table 6: Statistics regarding the PeerRead dataset [49]

These results can be observed in Table 7.

Even though it may seem unfair to compare one step of our two-step recommendation system against unified solutions, it is important to keep in mind that we could have simply used one of these alternatives as retrievers. Therefore, it is important to compare the retriever as is, especially since it dictates the performance cap of the re-ranker. To exemplify this, if we want to re-rank the top 50 results, the performance of our re-ranker is currently capped at the value of Recall@50 for every single Recall@k value of the re-ranker.

Model	R@5	R@10	R@30	R@50	R@80
Yang et al. [55]	0.21	0.27	0.41	0.48	0.55
BERT-GCN [8]	0.48	0.52	0.60	0.65	0.69
Our retriever	0.20	0.27	0.37	0.46	0.51

Table 7: Retriever performance on the PeerRead dataset

Though our approach is clearly outperformed by the BERT-GCN model of Jeong et al. [8], especially before re-ranking, the advantages of our model are worth the performance hit for our use case.

More specifically, the approaches used for comparison are very inflexible in terms of the set of papers they can recommend, meaning that they are limited to the papers they were trained on and modifications require retraining. Our retriever can adapt to new papers by just indexing some embeddings, offering excellent flexibility at the cost of some performance.

In terms of further research, it is clear that BERT-GCN [8] dominates due to its exploitation of the larger citation graph. We think the next logical step is finding a way to incorporate this information into our retriever to improve results.

Another approach that could improve performance is fine-tuning our embedder to this specific task. This could be done by measuring its current performance against Recall@k metrics. However, there is a possibility that this would lead to similar issues of inflexibility as neural network approaches, meaning the model could learn to put queries close to the set of papers used for training in vector space, while external papers would be farther away.

The evaluation metrics used for this task could also be improved. Currently, two models that

do not find the cited paper will score exactly the same, even if one of them has way more relevant recommendations compared to the other. Ideally, we'd have a way to compare sets of recommendations without needing the target paper to appear in them.

7.3 Re-ranking

To evaluate our re-ranker we used the top 50 candidate lists provided by retriever at the previous step and recalculated the Recall@k metrics after reordering, as seen in Table 4.

Evaluating the re-ranker using this method makes its performance be highly influenced by the quality of the retriever since the target paper won't always be in the list of candidates. Because of this, we produce a second list of candidates by inserting the cited paper in it if it's not already present. This way, we can evaluate both the recommendation system as a whole, as well as the re-ranker independently.

Model	R@5	R@10
Yang et al. [55]	0.21	0.27
BERT-GCN [8]	0.48	0.52
Our re-ranker (target paper always included)	0.49	0.60
Our re-ranker (candidates unmodified)	0.28	0.32

Table 8: Re-ranker performance on the PeerRead dataset

Unfortunately, these results mostly prove that our retriever is a bottleneck when it comes to matching the performance of other models, as our re-ranker is comparable to BERT-GCN [8] in performance if the target paper is guaranteed to be present in the retriever. Of course, this scenario isn't realistic even with a better retriever, but the performance gap is large enough for us to justify this conclusion.

Despite this, our recommendation system still has a respectable performance and comes with some very valuable benefits, as described previously in section 7.2, making it a better fit for expanding real world use cases which require high flexibility.

To improve re-ranking even further, we think different inputs should be explored as part of the reordering process to leverage the power of large language models to capture information through natural language. A couple of such inputs could be:

- the powerful information encoded in the citation graph
- paper metadata (journal, domain etc.)
- additional text² from the paper (introduction, state-of-the-art section etc.)

²Note: even though doing this would likely fill up the model's context window, our re-ranking method, RankGPT [42, 43], has mechanisms to offset this.

This could allow the re-ranker to better understand the context in which the target paper situates in, allowing it to offer better recommendations at the cost of additional inference time due to larger inputs.

7.4 Execution Time Analysis

We ran our pipeline on 10 different scientific papers with page counts between 15 and 20 to estimate the time it takes to execute on a single input paper. Our experiment resulted in a mean average runtime of 39.5 seconds with a standard deviation of 17.44 seconds, meaning that our method's execution time is quite volatile. This is likely due to the fact that the number of passages which require citation recommendation varies from paper to paper.

While the execution time is not extraordinary, it is fast enough for integration in most applications. The only notable exceptions are real-time use cases such as text editors, a scenario in which 40 seconds whenever we want to identify missing citations is not acceptable as execution could be required after common actions such as edits.

As a workaround, such integrations could opt to only apply the pipeline on the paragraphs that suffered modifications since the previous verification. This would reduce the length of the target text, speeding up overall inference time.

8 CONCLUSIONS

In this paper, we introduced an end-to-end pipeline for finding missing citations in papers, leveraging the transformer architecture [28], instructional text embeddings for document retrieval, as well as large language models for re-ranking and classification.

The end result is an all-in-one solution that doesn't require any input besides the target paper to determine sentences with missing citations, as well as recommend fitting scientific works to cite in the detected contexts.

We also achieved state-of-the-art performance on the citing sentence detection task [11], outperforming the current best solution, Wright et al.'s paragraph-level classifier [10], by 10 F1-score points.

The proposed recommendation system offers greater flexibility compared to alternative approaches, being capable of adapting to new sets of scientific papers that can be recommended by simply generating and indexing their embeddings in a database, no additional retraining needed. However, we did have to trade some performance for this benefit.

Our pipeline can easily be integrated into existing systems such as text editors or peer review platforms to mitigate the problem of missing citations in scientific publications, being fast enough for most use cases of scientific text analysis.

We think further improvements should exploit additional paper metadata, as well as the citation graph. These additions have the potential to improve both the citing sentence classifier and the recommendation, while keeping a similar degree of flexibility.

BIBLIOGRAPHY

- [1] Michael Fire and Carlos Guestrin. Over-optimization of academic publishing metrics: Observing goodhart’s law in action. *GigaScience*, 8, 06 2019.
- [2] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE Transactions on Neural Networks*, 20:61–80, 2009.
- [3] Seongjun Yun, Minbyul Jeong, Raehyun Kim, Jaewoo Kang, and Hyunwoo J. Kim. Graph transformer networks. In *Neural Information Processing Systems*, 2019.
- [4] Alan Lindsay Mackay. Publish or perish. *Nature*, 250:698–698, 1974.
- [5] Seema Rawat and Sanjay Kumar Meena. Publish or perish: Where are we heading? *Journal of Research in Medical Sciences : The Official Journal of Isfahan University of Medical Sciences*, 19:87 – 89, 2014.
- [6] Jialiang Lin, Yao Yu, Jia-Qi Song, and X. Shi. Detecting and analyzing missing citations to published scientific entities. *Scientometrics*, 127:2395 – 2412, 2022.
- [7] Patrick Lewis, Ethan Perez, Aleksandara Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. *ArXiv*, abs/2005.11401, 2020.
- [8] Chanwoo Jeong, Sion Jang, Hyuna Shin, Eunjeong Lucy Park, and Sungchul Choi. A context-aware citation recommendation model with bert and graph convolutional networks. *Scientometrics*, 124:1907 – 1922, 2019.
- [9] Qi He, Jian Pei, Daniel Kifer, Prasenjit Mitra, and Lee Giles. Context-aware citation recommendation. In *The Web Conference*, 2010.
- [10] Dustin Wright and Isabelle Augenstein. Citeworth: Cite-worthiness detection for improved scientific document understanding. In *Findings*, 2021.
- [11] Kazunari Sugiyama, T.Suresh Kumar, Min-Yen Kan, and Ramesh Chandra Tripathi. Identifying citing sentences in research papers using supervised learning. *2010 International Conference on Information Retrieval & Knowledge Management (CAMP)*, pages 67–72, 2010.
- [12] Frank Rosenblatt. The perceptron: a probabilistic model for information storage and organization in the brain. *Psychological review*, 65 6:386–408, 1958.

- [13] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Huggingface’s transformers: State-of-the-art natural language processing. *ArXiv*, abs/1910.03771, 2019.
- [14] James Reason. Human error: models and management. *BMJ*, 320 7237:768–70, 2000.
- [15] Arman Cohan, Bridger Waleed Ammar, Madeleine van Zuylen, and Field Cady. Structural scaffolds for citation intent classification in scientific publications. *ArXiv*, abs/1904.01608, 2019.
- [16] Yi Luan, Luheng He, Mari Ostendorf, and Hannaneh Hajishirzi. Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun’ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3219–3232, Brussels, Belgium, October-November 2018. Association for Computational Linguistics.
- [17] Avi Caciularu, Arman Cohan, Iz Beltagy, Matthew Peters, Arie Cattan, and Ido Dagan. CDLM: Cross-document language modeling. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 2648–2662, Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics.
- [18] Jakub Lála, Odhran O’Donoghue, Aleksandar Shtedritski, Sam Cox, Samuel G. Rodrigues, and Andrew D. White. Paperqa: Retrieval-augmented generative agent for scientific research. *arXiv preprint arXiv:2312.07559*, 2023.
- [19] David Jurgens, Srijan Kumar, Raine Hoover, Daniel A. McFarland, and Dan Jurafsky. Measuring the evolution of a scientific field through citation frames. *Transactions of the Association for Computational Linguistics*, 6:391–406, 2018.
- [20] Trevor Strohman, W. Bruce Croft, and David D. Jensen. Recommending citations for academic papers. In *Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2007.
- [21] Vladimir Naumovich Vapni. The nature of statistical learning theory. In *Statistics for Engineering and Information Science*, 2000.
- [22] Steven Bird, R. Dale, B. Dorr, Bryan R. Gibson, Mark Thomas Joseph, Min-Yen Kan, Dongwon Lee, Brett Powley, Dragomir R. Radev, and Yee Fan Tan. The acl anthology reference corpus: A reference dataset for bibliographic research in computational linguistics. In *International Conference on Language Resources and Evaluation*, 2008.

- [23] Michael Färber, Alexander Thiemann, and Adam Jatowt. To cite, or not to cite? detecting citation contexts in text. In *European Conference on Information Retrieval*, 2018.
- [24] Michael Färber, Alexander Thiemann, and Adam Jatowt. A high-quality gold standard for citation-based tasks. In *International Conference on Language Resources and Evaluation*, 2018.
- [25] Kyle Lo, Lucy Lu Wang, Mark Neumann, Rodney Michael Kinney, and Daniel S. Weld. S2orc: The semantic scholar open research corpus. In *Annual Meeting of the Association for Computational Linguistics*, 2020.
- [26] Iz Beltagy, Kyle Lo, and Arman Cohan. Scibert: A pretrained language model for scientific text, 2019.
- [27] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding, 2019.
- [28] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need, 2023.
- [29] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020.
- [30] OpenAI. Gpt-4 technical report. In *None*, 2023.
- [31] Google. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *ArXiv*, abs/2403.05530, 2024.
- [32] Xuewei Tang, Xiaojun Wan, and Xun Zhang. Cross-language context-aware citation recommendation in scientific articles. *Proceedings of the 37th international ACM SIGIR conference on Research & development in information retrieval*, 2014.
- [33] Travis Ebesu and Yi Fang. Neural citation network for context-aware citation recommendation. *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2017.
- [34] Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami, and Chris Dyer. Neural architectures for named entity recognition. In *North American Chapter of the Association for Computational Linguistics*, 2016.
- [35] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners, 2020.

- [36] Colin Raffel, Noam M. Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21:140:1–140:67, 2019.
- [37] Alvin Wei Ming Tan, Sunny Yu, Bria Long, Wanjing Anya Ma, Tonya Murray, Rebecca D. Silverman, Jason D. Yeatman, and Michael C. Frank. Devbench: A multimodal developmental benchmark for language learning, 2024.
- [38] Junchen Jiang, Vyas Sekar, Ion Stoica, and Hui Zhang. Unleashing the potential of data-driven networking. In *International Conference on Communication Systems and Networks*, 2017.
- [39] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Transactions on Information Systems*, 43:1 – 55, 2023.
- [40] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Annual Meeting of the Association for Computational Linguistics*, 2024.
- [41] Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Multilingual e5 text embeddings: A technical report. *ArXiv*, abs/2402.05672, 2024.
- [42] Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. Is chatgpt good at search? investigating large language models as re-ranking agent. *ArXiv*, abs/2304.09542, 2023.
- [43] Weiwei Sun, Zheng Chen, Xinyu Ma, Lingyong Yan, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Instruction distillation makes large language models efficient zero-shot rankers. *ArXiv*, abs/2311.01555, 2023.
- [44] J. Yablonski. *Laws of UX: Using Psychology to Design Better Products & Services*. O'Reilly Media, Incorporated, 2020.
- [45] Google. Gemini flash 2.0, 2024. Accessed: 2025-06-04.
- [46] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers. *ArXiv*, abs/1904.10509, 2019.
- [47] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *ArXiv*, abs/1907.11692, 2019.
- [48] Tairan Fu, Raquel Ferrando, Javier Conde, Carlos Arriaga, and Pedro Reviriego. Why do large language models (llms) struggle to count letters? *ArXiv*, abs/2412.18626, 2024.

- [49] Dongyeop Kang, Bridger Waleed Ammar, Bhavana Dalvi, Madeleine van Zuylen, Sebastian Kohlmeier, Eduard H. Hovy, and Roy Schwartz. A dataset of peer reviews (peerread): Collection, insights and nlp applications. *ArXiv*, abs/1804.09635, 2018.
- [50] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-pack: Packaged resources to advance general chinese embedding, 2023.
- [51] DeepSeek-AI. Deepseek-v3 technical report. *ArXiv*, abs/2412.19437, 2024.
- [52] Saurav Pawar, S. M Towhidul Islam Tonmoy, S M Mehedi Zaman, Vinija Jain, Aman Chadha, and Amitava Das. The what, why, and how of context length extension techniques in large language models - a detailed survey. *ArXiv*, abs/2401.07872, 2024.
- [53] H. M. and Sulaiman M.N. A review on evaluation metrics for data classification evaluations. *International Journal of Data Mining & Knowledge Management Process*, 5:01–11, 2015.
- [54] Meta. The llama 3 herd of models. *ArXiv*, abs/2407.21783, 2024.
- [55] Libin Yang, Yu Zheng, Xiaoyan Cai, Hang Dai, Dejun Mu, Lantian Guo, and Tao Dai. A lstm based model for personalized context-aware citation recommendation. *IEEE Access*, 6:59618–59627, 2018.